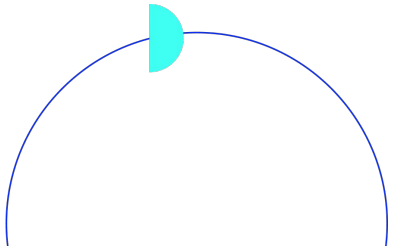
# Operaciones básicas

# Operaciones aritméticas

Todos los lenguajes de programación tienen la capacidad de **ejecutar operaciones matemáticas simples** (suma, resta, división, multiplicación), es decir, incorporan una suerte de calculadora virtual que podemos usar en nuestros programas para hacer este tipo de operaciones.

La **sintaxis** para realizar una suma en Python es muy similar a la que usamos en matemática para expresar dicha operación.

Por *sintaxis* entendemos **el orden y la relación de los términos de un lenguaje** necesarios para que una instrucción sea comprendida



En este ejemplo, podemos observar cómo realizar una **suma**.

```
>>> 7 + 5
12
```

Asimismo, como vemos en este caso, la **resta, multiplicación y división** se realizan mediante los operadores **-** (guión medio), ***** (asterisco) y **/** (barra invertida) respectivamente.

```
>>> 7 - 5
2

>>> 7 / 5
1.4

>>> 7 * 5
35
```



Los **objetos alrededor de un operador** (que en este ejemplo serían 7 y 5) pueden ser simples números enteros o de coma flotante, o bien expresiones que retornen un número entero o de coma flotante, o también variables que contengan un número entero o de coma flotante.

Analiza el ejemplo a la derecha:

```
>>> a = 5

>>> pi = 3.14

>>> a + 7
12

>>> pi - pi
0.0

>>> pi * 3
9.42

>>> a / pi
1.592356687898089
```

Estas operaciones aritméticas **retornan un entero** cuando los dos objetos sobre los que actúan son enteros.

Si alguno de ellos es un número de coma flotante, o bien ambos, el **resultado es un número de coma flotante**. De esto se sigue que `pi - pi` retorne `0.0` (un número de coma flotante) y no `0` (un entero).

También podemos hacer uso de los **paréntesis para agrupar expresiones**, como en matemática.

```
>>> 7 + 5 * 3
22

>>> (7 + 5) * 3
36
```



Para elevar (**potenciación**), se emplea el operador **** (doble asterisco).

```
>>> 2**3
8

>>> 4**2
16
```

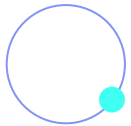
Con el operador **división entera** (//), el resultado que se devuelve es solo la parte entera.

```
>>> 5.5 // 2
1.0
```

El operador **módulo** **%** devuelve el resto de la división entre dos operandos.

```
>>> 9 % 2
1

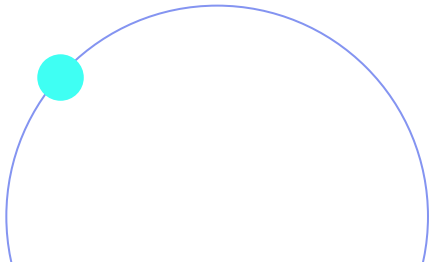
>>> 10 % 2
0
```



# Operaciones de comparación

Siguiendo con los números enteros y de coma flotante, además de las operaciones aritméticas, disponemos de **operaciones de comparación** mediante los siguientes operadores:

| | |
|---|---|
| == | igualdad |
| != | desigualdad |
| > | mayor |
| >= | mayor o igual |
| < | menor |
| <= | menor o igual |



Al ejecutar alguna de estas operaciones, **el resultado siempre es un booleano** (es decir, `True` o `False`), según la comparación sea verdadera o falsa, así como el resultado en las operaciones aritméticas siempre era un número entero o de coma flotante.

Observa estos ejemplos:

```
>>> 7 > 5
True

>>> 7 <= 5
False

>>> 7 != 5
True
```

```
>>> 7 == 5
False

>>> 5 >= 7
False

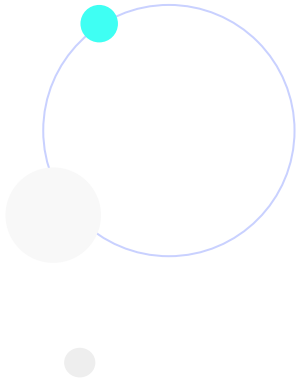
>>> 5 < 7
True
```



Los operadores **>**, **>=**, **<** y **<=** solo pueden aplicarse sobre números enteros o de coma flotante.

Los operadores de **igualdad (==) y desigualdad (!=)** nos permiten comparar de forma indistinta cualquiera de los cuatro tipos de dato básicos (y también otros que veremos más adelante).

Las **comparaciones** en general, y en particular las últimas dos que vemos en el ejemplo, son muy comunes en todo programa de Python.



```
>>> "Hola mundo" == 3.14
False
>>> 3.14 != True
True
>>> "Hola" != "mundo"
True
```

# Operaciones lógicas

Las operaciones lógicas son tan comunes como las comparaciones en cualquier programa, puesto que nos permiten **crear comparaciones más complejas**.

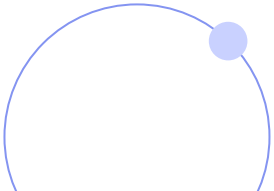
Consideremos la siguiente variable:

```
>>> a = 5
```

¿Cómo podemos determinar si **a** es mayor a 1 y menor a 10, o, lo que es lo mismo, si a se encuentra entre 1 y 10?

Necesitamos **combinar dos operaciones de comparación**. Para ello, empleamos la palabra reservada **and**. Esta operación se conoce como **conjunción**.

```
>>> a > 1 and a < 10
True
```



El **resultado** de una conjunción es verdadero cuando las dos condiciones que se propone conectar son verdaderas.

En este caso, `a > 1 and a < 10` es verdadero únicamente cuando `a > 1` y `a < 10` son ambas verdaderas. Si alguna de ellas es falsa, el resultado también lo es.

Confirmemos lo que acabamos de decir con el ejemplo que vemos a la derecha.

```
>>> True and True
True

>>> True and False
False

>>> False and TrueFalse
```



Por otro lado, la **disyunción** (**or**) nos permite saber si alguna de dos comparaciones es verdadera.

Una disyunción es verdadera cuando al menos una de las dos comparaciones que conecta es verdadera.

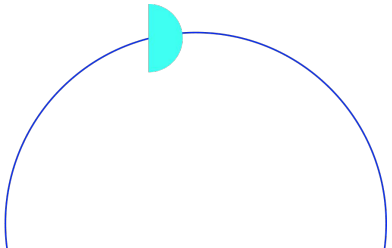
> Por ejemplo, queremos saber si `a` es igual a 1 o bien igual a 10, podemos seguir la segunda referencia a la derecha: aquí, tanto `a == 1` como `a == 10` son comparaciones falsas, puesto que `a` es 5.

```
>>> True or True
True

>>> True or False
True

>>> False or False
False
```

```
>>> a == 1 or a == 10
False
```



La **negación**, que se ejecuta con la palabra reservada **not**, invierte el valor lógico de un booleano.

```
>>> not True
False
>>> not False
True
```

Así, por ejemplo, las siguientes comparaciones son equivalentes:

```
>>> pi != 3.14
False
>>> not pi == 3.14
False
```



Al igual que en las operaciones aritméticas, podemos **agrupar expresiones logicas** usando paréntesis.

```
>>> a = 5
>>> pi = 3.14
>>> not pi == 3.14 and a == 1 or a == 10
False
>>> not (pi == 3.14 and a == 1 or a == 10)
True
```



## Tabla de verdad

Esta tabla de verdad resulta útil para comprender cada una de las operaciones que acabamos de describir.

| a | b | a and b | a or b | not a | not b |
|---|---|---|---|---|---|
| True | True | True | True | False | False |
| False | True | False | True | True | False |
| True | False | False | True | False | True |
| False | False | False | False | True | True |

# Antes de continuar...

# ¿Cómo imprimir los resultados?

**7 + 5** es un código de Python perfectamente válido, al igual que lo era **`print("Hola mundo")`**.

Modifiquemos nuestro programa **`hola.py`** para que contenga el siguiente código (recuerda que debemos mantener siempre las primeras dos líneas que mencionamos en el apartado “Nuestro primer programa”):

```
7 + 5
print("Hola mundo")
```

Al ejecutarse, este código realiza dos operaciones en el orden en que las dispusimos: primero ejecuta la suma **7 + 5** y luego imprime en pantalla el mensaje **“Hola mundo”**.

¡No esperes ver el resultado de la suma en pantalla! No, **Python no hace nada que no le hayamos indicado**; y, en efecto, simplemente le hemos dicho “resuelve 7 + 5”, pero no hicimos mención sobre qué debe hacer con ese resultado.

Vayamos un poco más allá, entonces, y digámosle que **imprima el resultado** en la pantalla:

```
print(7 + 5)
print("Hola mundo")
```

Ahora sí, al ejecutar ese código, veremos:

```
12
Hola mundo
```

Todo lo que pongamos **dentro del paréntesis** en una instrucción `print` será enviado a la consola.



Ahora bien, ¿por qué utilizamos comillas en el segundo caso y no en el primero? Por el momento, basta con saber que **las comillas permiten que un conjunto de caracteres sea interpretado de forma literal** y no como código de Python.

Teniendo esto en cuenta, veamos el siguiente código y su resultado:

```
print("7 + 5")
print("Hola mundo")
```

--------►

```
7 + 5
Hola mundo
```



En este caso, **"7 + 5"** es **interpretado como un texto** en lugar de una operación que Python deba ejecutar, puesto que está encerrado entre comillas.

La función **`print`** puede tomar **más de un argumento de cualquier tipo**, separados por coma. Esta función genera la salida con todos los argumentos **convertidos a `str` y concatenados**, incluyendo un espacio entre ellos. Por ejemplo:

```
print("Total:", 5 + 2)
Total: 7
```

En este caso, `print` toma dos argumentos: la cadena `"Total:"` y el resultado de la operación 5 + 2, que es 7. La función convierte ambos a **cadenas de texto** y los concatena con un espacio entre ellos, produciendo la salida `Total: 7`.

